☀ SmartSolar / SolarCast — Capstone Project Summary

For Board Presentation

◇ 1. What Is This Project?

SmartSolar (SolarCast) is a Machine Learning–powered solar energy prediction system built as a university capstone project. Its purpose is to answer two critical questions that every solar panel owner or installer faces:

> **"How much power will my solar panels actually produce right now?" "How long will my battery last if grid power goes out?"**

The system goes beyond simple textbook formulas by using **7 trained Machine Learning models** to predict real-world output — accounting for dirt, aging, shading, and wiring inefficiencies that standard physics equations cannot capture.

◇ 2. The Problem We're Solving

The Physics Formula Alone Is Not Enough

There is a standard engineering formula for solar panel power:

```
Power = Efficiency × Panel Area × Sunlight × Temperature Correction
```

While this formula looks good on paper, **in the real world it always overestimates actual output** because it ignores:

Real-World Loss	Cause	Typical Impact
▧ **Soiling / Dust**	Dirt, bird droppings, pollution on glass	5–20% loss
⌛ **Panel Aging**	Degradation over time (~0.5%/year)	Up to 8% loss
🌳 **Partial Shading**	Trees, buildings blocking part of a panel	15–50% loss
🔌 **Wiring & Mismatch**	Cable resistance + panel-to-panel variation	1–5% loss

The gap between the formula's prediction and real output is exactly what our ML models learn to close.

◇ 3. Our Solution — The Two-Stage System

The project is divided into two components:

Stage 1: The ML Training Pipeline (`solar_pipeline.py`)

This is the "engine room" — it runs once to generate data, train all models, and save results.

Stage 2: The Live Dashboard (`app.py`)

This is the interactive web application — it uses the trained models to give instant predictions based on user inputs.

◇ 4. How the Data Was Built

Since real sensor data wasn't available, we **synthetically generated a realistic dataset of 50,000 data points** using physics-based simulation with real-world randomness added.

What each data point includes:

- **Weather**: Sunlight intensity, temperature, humidity, wind speed
- **Time**: Hour of day, month, day of year (for seasonal effects)
- **Panel specs**: Efficiency, panel area, temperature coefficient, NOCT
- **Target label**: Actual DC power output — calculated using the physics formula **multiplied by all four real-world loss factors**

Why this approach is academically sound:

The ML models cannot simply "memorise" the physics formula — the loss factors introduce **real unpredictability** that only a data-driven model can learn. This is what makes ML genuinely necessary here.

◇ 5. The 7 Machine Learning Models

We trained and compared **7 different ML algorithms** on the same dataset:

#	Model	Type	Strength
1	**Random Forest**	Ensemble (trees)	Robust, high feature importance visibility
2	**Gradient Boosting**	Ensemble (sequential)	Strong general accuracy
3	**MLP / ANN**	Neural Network	Captures complex non-linear patterns
4	**XGBoost**	Gradient boosting (optimised)	Industry-standard, very fast
5	**LightGBM**	Gradient boosting (light)	Highly efficient on large data
6	**SVR (RBF)**	Support Vector Machine	Good for smooth, non-linear relationships
7	**Extra Trees**	Ensemble (random splits)	Fast training, low variance

How Models Are Evaluated (4 Metrics):

- **R^2** — How much of the variation in output the model explains (higher = better, max = 1.0)
- **RMSE** — Root Mean Square Error in Watts (lower = better)

- **MAE** — Mean Absolute Error in Watts (lower = better)
- **MAPE** — Mean Absolute Percentage Error, measured only on outputs > 50W (avoids inflated errors at dawn/dusk when output is near zero)

> [!IMPORTANT] The final dashboard uses an **ensemble average** of all 7 models — not just one winner. This is more robust than any single model and reduces prediction risk.

◇ 6. The Training Pipeline — Step by Step

```
STEP 1 → Generate 50,000 synthetic data points with real-world loss factors
STEP 2 → Exploratory Data Analysis (EDA) — visualise patterns and correlations
STEP 3 → Generate pipeline architecture diagram
STEP 4 → Preprocessing — scale features, split 80% train / 20% test
STEP 5 → Train all 7 models simultaneously
STEP 6 → Evaluate each model with 4 metrics, generate comparison charts
STEP 7 → Save all trained models as .pkl files for use in the dashboard
```

Files saved after training:

File	What it is
`models/rf_model.pkl`	Trained Random Forest
`models/gb_model.pkl`	Trained Gradient Boosting
`models/mlp_model.pkl`	Trained Neural Network
`models/xgb_model.pkl`	Trained XGBoost
`models/lgb_model.pkl`	Trained LightGBM
`models/scaler.pkl`	Feature normaliser
`models/best_model.pkl`	Copy of the top-performing model

◇ 7. The Live Dashboard — SmartSolar

The dashboard is a fully interactive web application built with **Streamlit** and **Plotly**. It runs in the browser and lets anyone configure their solar setup and get instant predictions.

Sidebar Inputs (what the user controls):

Section	Parameters
Panel Preset	Choose from 11 real panels (Jinko, Canadian Solar, SunPower, etc.) or custom
Panel Specs	Rated Wp, efficiency %, area, temp coefficient, NOCT, number of panels
Real-World Losses	Soiling %, aging %, wiring %, shading % (sliders)
System Specs	Inverter efficiency, battery capacity, depth of discharge, load demand

Section	Parameters
Weather	Irradiance, temperature, humidity, wind speed, hour, month

What the Dashboard Shows:

Live Predictions Row

- Individual W prediction from each of the 7 models
- **ML Ensemble** (average of all 7 models)
- **Adjusted Output** (ensemble × real-world loss correction)
- **Backup Runtime** (hours of battery backup at your load)

Real-World Loss Breakdown

- Side-by-side comparison: Physics formula vs actual adjusted output
- Shows exactly how many Watts each loss type costs

Visual Charts (Output Gauge + Module Temperature + Backup Bar Chart)

- Colour-coded gauge: Low / Moderate / Good
- Live module temperature with NOCT heating model
- Energy breakdown: Panel Wh + Battery Wh = Total available

5 Analysis Tabs:

Tab	What it shows
Today's Profile	Hourly power curve for the whole day
Irradiance Sensitivity	How output changes as sun intensity varies 0–1100 W/m^2
Temperature Sensitivity	How output drops as temperature rises
Formula vs ML	Scatter plot proving ML outperforms the physics formula
Model Performance	Full metrics table + R^2 bar chart + feature importance chart

◇ 8. Key Technical Decisions Explained

☑ Why ML instead of just the formula?

The physics formula **consistently overestimates** real output because it cannot model soiling, aging, shading, or wiring losses. Our training data proves this — the scatter plot in the dashboard shows all physics formula predictions scatter **above** the "perfect prediction" diagonal line, meaning the formula always over-predicts.

☑ Why Ensemble (average of all 7 models)?

Using all 7 models and averaging reduces the risk that any single model's weakness affects the result. If one model overfits or has a blind spot, the others compensate.

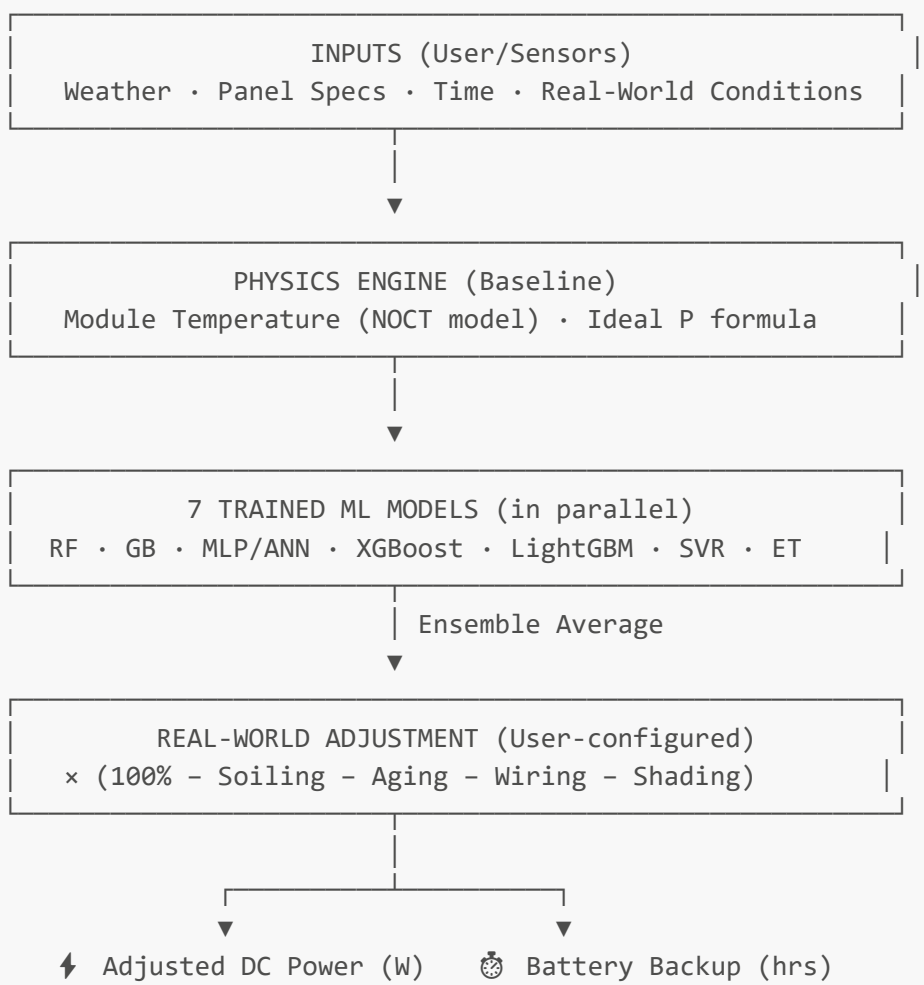
☑ Why MAPE threshold > 50W?

At dawn and dusk, panels produce only 2–10 W. Even a good prediction of 5W vs actual 3W gives 67% error — technically correct but misleadingly alarming. We restrict MAPE calculation to outputs above 50W, which are the only hours relevant for energy planning.

☑ Why 50,000 training samples?

Enough to cover the full range of panel types (14%–22% efficiency), weather conditions, seasons, and shading scenarios — making the model generalise to any real-world installation.

◇ 9. System Architecture Overview



◇ 10. Project Files Summary

File / Folder	Purpose
`solar_pipeline.py`	Full ML training pipeline (run once)
`app.py`	Live Streamlit dashboard (run always)
`data/solar_dataset.csv`	50,000-row training dataset (~4 MB)

File / Folder	Purpose
`models/*.pkl`	7 trained model files + scaler
`outputs/*.png`	EDA charts, model comparison, pipeline diagrams

◇ 11. What Makes This Project Stand Out

> [!NOTE] This is not a simple "fit a model to a dataset" project. Key innovations:

1. **Physics-informed ML** — the formula provides a transparent baseline; ML adds real-world intelligence on top
2. **Multi-model ensemble** — reduces prediction risk vs relying on a single algorithm
3. **Four independent loss factors** — makes ML genuinely necessary (formula alone cannot reproduce training labels)
4. **Production-ready dashboard** — any user can input their real panel specs and get personalised predictions
5. **Academically honest metrics** — MAPE threshold corrected to avoid misleading dawn/dusk inflation

---

◇ 12. One-Sentence Summary for the Board

> **SmartSolar uses 7 machine learning models trained on 50,000 realistic data points to predict the actual output of any solar panel system — accounting for dust, aging, shade, and wiring losses that standard engineering formulas miss — and delivers this as an interactive web dashboard with real-time battery backup calculations.**